

ESCUELA COLOMBIANA DE INGENIERÍA
DESARROLLO ORIENTADO POR OBJETOS
PROYECTO INICIAL 2026-2
Ciclo No. 2/4 — Refactoring y Extensión

El proyecto inicial tiene como propósito desarrollar una aplicación que permita simular una situación inspirada en el **Problem I** de la maratón de programación internacional **2025 Slot Machine**. En este simulador los símbolos deben ser de colores diferentes.

SEGUNDO CICLO

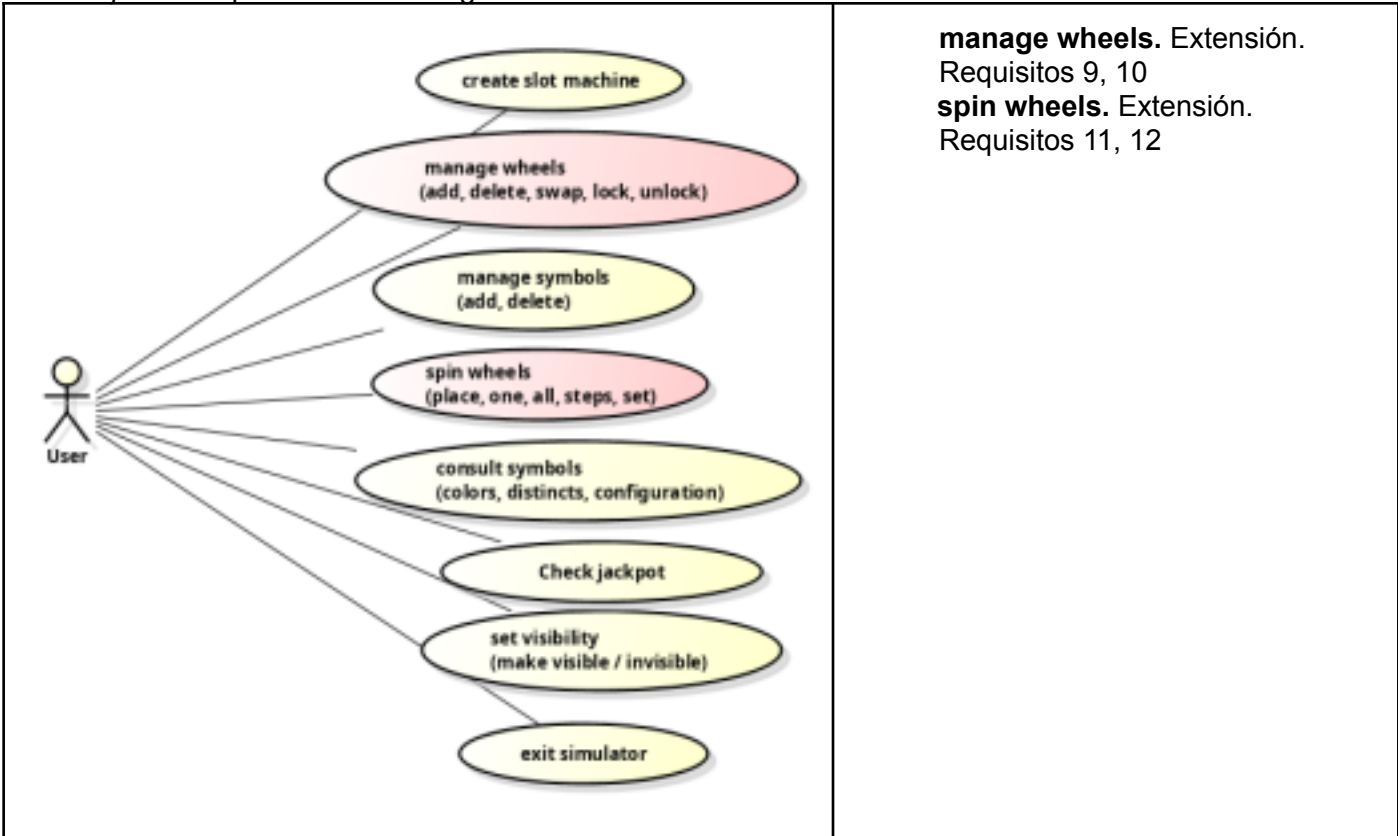
Los requisitos para este ciclo de desarrollo están indicados a continuación. Siempre hay un requisito implícito: el de **EXTENSIBILIDAD**.

MUY IMPORTANTE PLANIFICAR LOS MINI-CICLOS PARA ORIENTAR EL DESARROLLO. En esta entrega **NO** deben resolver el problema de la maratón sólo deben construir el simulador.

REQUISITOS FUNCIONALES

El simulador debe permitir:

- 9. Intercambiar dos ruedas
- 10. Fijar y soltar una rueda
- 11. Rotar una rueda un número de pasos
- 12. Dejar la máquina en una configuración dada



REQUISITOS DE DISEÑO

SlotMachine
+ SlotMachine() + addWheel(pos : int) : void + delWheel(pos : int) : void + swap(wheel1 : int, wheel2 : int) : void + lock(wheel : int) : void + unlock(wheel : int) : void + addSymbol(pos : int, color : String) : void + delSymbol(symbol : String) : void + placeSymbol(wheel : int, symbol : String) : void + spin(wheel : int) : void + spin(wheel : int, steps : int) : void + spin(setSymbols : String[]) : void + spin() : void + symbols() : String[] + distinctSymbols() : int + configuration() : String[] + isJackpot() : boolean + makeVisible() : void + makeInvisible() : void + exit() : void + ok() : boolean

REQUISITOS DE USABILIDAD

1. En [spin\(wheel,steps\)](#), si está visible, el movimiento debe visualizarse paso a paso

REQUISITOS DE ENTREGA

Los productos de esta entrega son:

1. Diseño completo en la herramienta astah
2. Implementación siguiendo los estándares de codificación y documentación de java.
3. Casos de pruebas de unidad de su proyecto: [SlotMachineC2Test](#)

Las pruebas de unidad deben ser en modo invisible.

No olviden diseñar las pruebas considerando dos preguntas: ¿qué debería hacer? ¿qué no debería hacer? 4. Mínimo dos casos de prueba de unidad compartidos en la clase [SlotMachineCC2Test](#) Las clase [SlotMachineCC2Test](#) será una creación colectiva usando el wiki correspondiente. Los nombres de los casos de prueba deberán incluir la identificación de los autores.

Por ejemplo, [accordingDcAvShould...](#)

(DcAv: Iniciales de los primeros apellidos con primera letra del segundo en orden alfabético).

5. Dos pruebas de aceptación preparadas para la presentación.

6. Documento de retrospectiva del proyecto. (Ver ciclo uno)

Es necesario incluir la retrospectiva de este ciclo y de los anteriores.

DEFINIR EL ESTADO EN TÉRMINOS DE LOS MINI-CILOS PLANIFICADOS

#Pregunta	Pregunta	Ciclo1	Ciclo 2
1.	¿Cuáles fueron los mini-ciclos definidos? Justifíquenlos.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.	Los mini-ciclos definidos se pueden evidenciar en las partes siguientes de este documento.
2.	¿Cuál es el estado actual del proyecto en términos de mini-ciclos? ¿por qué?	Completo	Completo
3.	¿Cuál fue el tiempo total invertido por cada uno de ustedes? (Horas/Hombre)	15 horas cada uno	18 horas cada uno
4.	¿Cuál consideran fue el mayor logro? ¿Por qué?	Aplicar la práctica XP llamada Pair Programming	Aplicar la práctica TDD porque está al aplicarse brinda cierta seguridad en que las funcionalidades que se exigen en el proyecto se cumplan.
5.	¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?	Utilizar GitHub, esto se fue aprendiendo a medida que se utilizaba para guardar el proyecto.	Entender los requerimientos que se pedían, lo que se hizo fue hablar con la profesora para asegurar lo que se creía de cada requerimiento.
6.	¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?	Comunicarnos	Lo que pudimos hacer fue redistribuirnos trabajo y explicarnos de manera sencilla lo que hizo cada uno.
7.	Considerando las prácticas XP incluidas en los laboratorios. ¿cuál fue la más útil? ¿por qué?		La comunicación con el cliente o profesor fue muy necesaria para cumplir los requisitos y esta es una de las prácticas XP, estar en constante diálogo con el cliente
8.	¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.	Referencias: Google. (2026). <i>Gemini</i> (Gemini Flash 3.6 y Pro) [Modelo de lenguaje de gran tamaño]. https://gemini.google.c	Referencias: Martin, R. C. (2012). <i>Código limpio: Manual de estilo para el desarrollo ágil de software</i> (2.ª ed.). Anaya Multimedia.

		<u>om</u> Anthropic. (2026). <i>Claude</i> (Versión [indicar versión, ej. Claude 3.6 Sonnet]) [Modelo de lenguaje de gran tamaño]. https://claude.ai	Google. (2026). <i>Gemini</i> (Gemini Flash 3.6 y Pro) [Modelo de lenguaje de gran tamaño]. https://gemini.google.com Anthropic. (2026). <i>Claude</i> (Versión [indicar versión, ej. Claude 3.6 Sonnet]) [Modelo de lenguaje de gran tamaño]. https://claude.ai
--	--	---	--

Desarrollo del ciclo 1:

#Miniciclo	SlotMachine	Wheel	Symbol
1	☒ SlotMachine() ☒ addWheel(int pos) ☒ delWheel(int pos) ☒ addSymbol(int pos, String color) ☒ delSymbol(String symbol)	☒ Wheel() ☒ changeColor(color: String) ☒ changePositionX(pos: int) ☒ changePositionY(pos: int)	☒ Symbol(color: string) ☒ changePositionX(pos: int) ☒ changePositionY(pos: int)
2	☒ symbols() ☒ placeSymbol(int wheel, String symbol) ☒ distinctSymbols() ☒ configuration()	☒ addSymbol(Symbol: String) ☒ delSymbol(Symbol: String) ☒ selectSymbol()	☒ makeVisible() ☒ makeInvisible()
3	☒ spin(int wheel) ☒ spin() ☒ isJackpot() ☒ configuration() ☒ ok() ☒ exit()	☒ spin() ☒ symbols()	
4	☒ makeVisible() ☒ makeInvisible()	☒ makeVisible() ☒ makeInvisible()	

Desarrollo del ciclo 2:

#Miniciclo	SlotMachine	Wheel	Symbol
1	☒ swap(wheel1 : int, wheel2 : int) ☒ lock(wheel : int) ☒ unlock(wheel : int)	☒ Agregar atributo isLocked y agregar lógica de spin	
2	☒ spin(wheel : int, steps : int) ☒ spin(setSymbols : String[])	☒ Spin(steps:int) ☒ spinSlowly(steps:int)	

Los productos los deben publicar en un repositorio Git y deben publicar en Moodle un archivo txt que contenga la dirección URL. El nombre del archivo .txt debe ser la concatenación de los primeros apellidos más la primera letra del segundo apellido de los autores, ordenados alfabéticamente.

Publicar productos a revisión: Semana 06: 5 septiembre